

IMPLEMENTACIÓN DEL PROTOTIPO

SABOR & GESTIÓN

Manual de Instalación, Configuración y Puesta en Funcionamiento del Backend

Guía Metodológica para Reproducción de Entornos, Arquitectura en Capas y Operación Transaccional

METADATOS DEL PROYECTO Y COMPONENTE BACKEND

Componente Analizado:	Backend RESTful API & Servidor de WebSockets
Repositorio Oficial:	sabor-gestion-backend-personal
Rama de Referencia:	main (Estado Post-Auditoría Fase 17.6)
Lenguaje y Runtime:	TypeScript 5.8.3 / Node.js >=20.x
Persistencia Transaccional:	MongoDB Atlas / Mongoose 9.3.1 (11 Colecciones)
Servicio de Tiempo Real:	Socket.IO 4.8.3 (Salas y Handshake JWT)
Gestor de Paquetes:	pnpm 12.4.2 (Lockfile Determinista)

Autor / Desarrollador:

Fernando Banda Téllez

Octubre 2026

TABLA DE CONTENIDOS

1.	Introducción	3
2.	Descripción General del Prototipo	3
3.	Requisitos del Entorno de Ejecución	4
4.	Instalación del Proyecto Paso a Paso	5
5.	Configuración del Entorno (.env)	7
6.	Configuración de Servicios Externos	8
7.	Estructura y Organización del Código	9
8.	Implementación Funcional por Módulos (M01 – M10)	10
9.	Ejecución del Backend en Modo Desarrollo y Producción	12
10.	Prueba Inicial del Sistema (Guía de Verificación)	13
11.	Integración y Comunicación entre Componentes	15
12.	Medidas de Seguridad y Reglas de Integridad	16
13.	Estrategia y Configuración de Despliegue	17
14.	Mantenimiento y Reproducción del Entorno	17
15.	Conclusiones	18

Propósito del Manual Técnico

Este documento constituye la memoria técnica formal de la sección "10. Implementación del Prototipo". Ha sido diseñado para permitir que cualquier evaluador académico o desarrollador de software pueda tomar el repositorio oficial, comprender su estructura de capas, instalar sus dependencias de forma determinista, configurar sus credenciales y ponerlo en marcha sin depender de asistencia externa.

1. INTRODUCCIÓN

La presente sección documenta la materialización técnica e implementación del prototipo para el sistema de información gastronómico "Sabor & Gestión". En el marco de la ingeniería de software moderna, la fase de implementación no se limita a la escritura de código fuente, sino que abarca la configuración de entornos reproducibles, la estructuración de la persistencia de datos, el gobierno de dependencias, la protección criptográfica y la definición de procedimientos de puesta en marcha confiables.

El componente central desarrollado e implementado en este repositorio corresponde al Backend: un núcleo transaccional desacoplado que expone una Interfaz de Programación de Aplicaciones (API RESTful) complementada por un canal de comunicación bidireccional en tiempo real basado en WebSockets (Socket.IO). El propósito fundamental de este manual es eliminar cualquier ambigüedad técnica, proveyendo instrucciones rigurosas, reproducibles y verificadas para desplegar y operar el sistema gastronómico a partir exclusivamente de los archivos contenidos en el repositorio.

2. DESCRIPCIÓN GENERAL DEL PROTOTIPO

El backend de Sabor & Gestión actúa como el cerebro transaccional y el árbitro de negocio de toda la operativa del restaurante. Centraliza el control de acceso, la administración de personal, la catalogación de la oferta gastronómica, el monitoreo culinario de inventarios, la rotación de mesas en vivo, el seguimiento de comandas en fogones y la liquidación financiera autoritativa.

2.1. Alcance y Capacidades Funcionales

El sistema articula 10 capacidades funcionales operativas fuertemente integradas:

- **Autenticación y Seguridad:** Control de acceso basado en roles (RBAC) con credenciales encriptadas vía bcryptjs (10 salt rounds) y emisión de JSON Web Tokens (JWT) con 8 horas de vigencia.
- **Gestión de Menú y Multimedia:** Catalogación jerárquica de categorías y platos con precio oficial congelado y almacenamiento en la nube mediante Cloudinary.
- **Disponibilidad Culinaria Preventiva:** Validación de ingredientes e insumos previa a la admisión de comandas, con exclusión semántica por notas (ej. "sin cebolla").
- **Gestión de Salón y Mesas:** Administración de ubicaciones físicas y mesas con control estricto de ocupación (Libre, Ocupada, Cuenta Solicitada) sincronizado en vivo.
- **Ciclo de Vida de Comandas:** Generación correlativa atómica PED-XXXX, inmutabilidad de precios unitarios y reapertura automática ante la adición de nuevos platos.
- **Seguimiento en Cocina:** Máquina de estados finita (ABIERTO -> EN_PREPARACION -> ENTREGADO) con avisos sonoros instantáneos hacia los meseros.
- **Liquidación Financiera Autoritativa:** Cálculo server-side del total (subtotal - descuento + propina, sin IVA) con ejecución en transacción atómica multi-documento de MongoDB.
- **Comprobantes y Pagos QR:** Emisión de facturas digitales HTML vía EmailJS y soporte de simulación pedagógica de pagos móviles con QRServer API.
- **Cierre de Turnos y Dashboard:** Arqueo automático de balances en CierreCaja al desactivar cajeros y analítica gerencial con 8 KPIs calculados en tiempo real.

2.2. Delimitación Estricta y Decisiones de Diseño

Para preservar la coherencia y robustez del prototipo, se establecieron exclusiones deliberadas de alcance: el sistema es estrictamente B2B (para uso del personal en salón, cocina y caja, sin portal para comensales externos); no utiliza Google OAuth (las cuentas son auditadas centralmente por el Administrador); no gestiona reservas de fechas futuras (la rotación se gestiona en tiempo real); no incluye módulo de reparto/delivery; y el flujo QR es una simulación pedagógica que requiere confirmación en caja.

3. REQUISITOS DEL ENTORNO DE EJECUCIÓN

Antes de proceder con la instalación y ejecución del backend, el equipo o servidor anfitrión debe cumplir con los requisitos de infraestructura y software base descritos a continuación. Todos los componentes han sido seleccionados garantizando compatibilidad multiplataforma (Linux, macOS y Microsoft Windows).

Herramienta / Entorno	Versión Requerida	Propósito en el Prototipo	Comando de Verificación
Node.js	>= 20.x LTS	Motor de ejecución asíncrono sobre el motor V8 de Google Chrome. Permite la ejecución del servidor TypeScript/JavaScript.	node -v
pnpm	12.4.2 (>= 8.x)	Gestor de paquetes determinista de alta velocidad. Gestiona las dependencias mediante almacenamiento global y enlaces duros.	pnpm -v
MongoDB	>= 6.0 (Atlas)	Base de datos NoSQL orientada a documentos. Requiere soporte para Replica Set / transacciones atómicas de sesión.	mongosh --version
Git	>= 2.x	Sistema de control de versiones para la clonación y sincronización del repositorio oficial.	git --version
TypeScript	5.8.3 (^5.0)	Compilador estricto para transpilación hacia JavaScript y verificación de interfaces de dominio (incluido en dependencias).	pnpm tsc -v

3.1. Procedimiento de Verificación en Terminal

Ejecute la siguiente secuencia en la terminal para constatar que las herramientas base se encuentran operativas:

Terminal — Comprobación de Requisitos Previos

```
# Verificación de la versión de Node.js (Debe ser v20.x o superior)
$ node --version
v20.18.0

# Verificación del gestor de paquetes pnpm
$ pnpm --version
12.4.2

# Verificación del cliente Git
$ git --version
git version 2.43.0
```

3.2. Instalación de pnpm

El proyecto utiliza formalmente pnpm como gestor oficial, declarado en package.json mediante "packageManager": "pnpm@12.4.2". Si su sistema no cuenta con pnpm, puede habilitarlo de manera nativa mediante Corepack (incorporado en Node.js) o mediante npm global:

Terminal — Habilitación de pnpm

```
# Opción A: Habilitación nativa mediante Corepack (Recomendado)
$ corepack enable
$ corepack prepare pnpm@12.4.2 --activate

# Opción B: Instalación global mediante npm convencional
$ npm install -g pnpm@12.4.2
```

Importancia del Gestor pnpm

No se recomienda utilizar npm convencional ni yarn para instalar las dependencias de este proyecto. El archivo pnpm-lock.yaml garantiza que se resuelvan exactamente las mismas subdependencias y árboles de compilación probados durante la fase de auditoría y hardening.

4. INSTALACIÓN DEL PROYECTO PASO A PASO

El procedimiento de instalación se ha estructurado de forma secuencial y limpia. A continuación se describe cada fase requerida para desplegar el árbol de trabajo localmente.

4.1. Clonación del Repositorio Oficial

Abra una ventana de terminal y ejecute el comando git clone indicando la URL remota del proyecto. Esto descargará la rama principal (main) con todo el código fuente y especificaciones técnicas:

Paso 1 — Clonar Repositorio

```
$ git clone https://github.com/BandaTellezFernando/sabor-gestion-backend-personal.git
Cloning into 'sabor-gestion-backend-personal'...
remote: Enumerating objects: 1240, done.
remote: Counting objects: 100% (1240/1240), done.
Receiving objects: 100% (1240/1240), done.
```

4.2. Acceso al Directorio Raíz

Ingresa a la carpeta del proyecto recién descargada:

Paso 2 — Directorio de Trabajo

```
$ cd sabor-gestion-backend-personal
```

4.3. Instalación de Dependencias

Ejecute el comando de instalación de pnpm. Este proceso leerá el archivo pnpm-lock.yaml y creará la carpeta node_modules vinculando los paquetes mediante enlaces duros sin duplicación de almacenamiento:

Paso 3 — Descarga e Instalación de Librerías

```
$ pnpm install
Resolving: total 27 dependencies, 11 devDependencies
Lockfile is up to date, resolution step is skipped
Packages: +258
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
Progress: resolved 258, reused 258, downloaded 0, added 258, done

dependencies:
+ bcryptjs 3.0.3
+ clouddinary 2.9.0
+ cors 2.8.6
+ dotenv 17.3.1
+ express 5.2.1
+ jsonwebtoken 9.0.3
+ mongoose 9.3.1
+ morgan 1.10.1
+ multer 2.1.1
+ socket.io 4.8.3
```

4.4. Verificación de Integridad de la Instalación

Para verificar que todos los módulos y dependencias de desarrollo (compilador TypeScript, tipos de Node, ESLint, nodemon) se instalaron adecuadamente, ejecute:

Paso 4 — Auditoría de Dependencias Instaladas

```
$ pnpm list --depth=0
Legend: production dependency, optional dependency, dev dependency

sabor-gestion-backend@1.0.0 /home/fercho/Software/sabor-gestion-backend-personal

dependencies:
bcryptjs 3.0.3      dotenv 17.3.1      morgan 1.10.1
clouddinary 2.9.0  express 5.2.1      multer 2.1.1
cors 2.8.6          jsonwebtoken 9.0.3 socket.io 4.8.3
mongoose 9.3.1
```

```
devDependencies:
@types/node 20.19.0  nodemon 3.1.14      tsx 4.21.0
eslint 8.57.1      prettier 3.6.2      typescript 5.8.3
ts-node 10.9.2
```

5. CONFIGURACIÓN DEL ENTORNO (.env)

El backend utiliza la biblioteca dotenv para desacoplar las credenciales y parámetros de configuración del código fuente. Por estrictos principios de seguridad de la información, el archivo .env se encuentra registrado en el archivo .gitignore y jamás debe ser confirmado ni subido a repositorios de control de versiones.

5.1. Creación del Archivo de Entorno

En la raíz del proyecto, cree un archivo titulado .env:

Terminal — Creación del Archivo de Entorno

```
$ touch .env
```

5.2. Catálogo Exhaustivo de Variables de Entorno Reales

A continuación se detallan las 11 variables de entorno que el código fuente inspecciona mediante process.env. Se suministran valores de ejemplo completamente ficticios y seguros:

Variable	Propósito en el Backend	Obligatoriedad	Ejemplo Seguro
PORT	Puerto TCP del servidor HTTP y WebSockets	No (Def: 3000)	3000
NODE_ENV	Modo de ejecución (development/production). Protege seeds	Sí	development
MONGO_URI	URI de conexión a MongoDB Atlas (usuario, clave y clúster)	Sí (Crítica)	mongodb+srv://user_dev:pass_sec@cluster.mongodb.net/sabor
JWT_SECRET	Clave simétrica para firmar/verificar tokens JWT (HS256)	Sí (Crítica)	clave_secret_jwt_ficticia_produccion_2026
CLOUDINARY_CLOUD_NAME	Nombre de la nube en Cloudinary para multimedia	Sí (Imágenes)	cloud_ficticio_gastronomia
CLOUDINARY_API_KEY	Llave pública de autenticación con Cloudinary	Sí (Imágenes)	123456789012345
CLOUDINARY_API_SECRET	Secreto privado de la cuenta de Cloudinary	Sí (Imágenes)	abcdefghijklmnopqrstuvwxyz123_ficticio
EMAILJS_SERVICE_ID	ID del servicio en EmailJS para despacho de recibos	Sí (Recibos)	service_ficticio_recibos
EMAILJS_TEMPLATE_ID	ID de la plantilla HTML configurada en EmailJS	Sí (Recibos)	template_ficticio_factura
EMAILJS_USER_ID	Llave pública / User ID de EmailJS	Sí (Recibos)	user_public_ficticio_123
EMAILJS_ACCESS_TOKEN	Token privado para autorización de llamadas REST	Sí (Recibos)	priv_token_ficticio_emailjs

5.3. Impacto Operativo de Variables Ausentes

El código implementa defensas programáticas que reaccionan ante la omisión de variables:

- **MONGO_URI Ausente:** El método connectDB() en src/configs/db.ts detecta el valor indefinido, imprime un mensaje de error y detiene el proceso con process.exit(1).
- **JWT_SECRET Ausente:** Los métodos de inicio de sesión (usuario.service.ts) y de autenticación en Socket.IO (socket.ts) rechazan inmediatamente las solicitudes ("FATAL ERROR: JWT_SECRET no configurado").
- **Cloudinary Ausente:** El endpoint POST /api/upload retorna error HTTP 500 y no procesa la imagen.
- **EmailJS Ausente:** El endpoint POST /api/pagos/:pedidold/enviar-recibo maneja la falla sin abortar el cobro previo.

6. CONFIGURACIÓN DE SERVICIOS EXTERNOS

El backend integra cuatro servicios externos para cubrir la persistencia transaccional, la entrega de imágenes, el despacho de comprobantes y la generación de códigos QR. Cada servicio debe configurarse como se indica a continuación:

6.1. MongoDB Atlas (Base de Datos Transaccional)

1. Función: Almacena de forma persistente y concurrente las 11 colecciones documentales del restaurante.
2. Variables requeridas: MONGO_URI.
3. Pasos del desarrollador: Crear una cuenta gratuita en MongoDB Atlas, aprovisionar un clúster M0 (Shared), crear un usuario con privilegios de lectura/escritura (readWriteAnyDatabase) y en Network Access agregar la IP de desarrollo (o 0.0.0.0/0 para pruebas).
4. Comprobación: Al iniciar el backend, la consola debe exhibir: "Base de Datos MongoDB Conectada: <nombre_cluster>".

6.2. Cloudinary (Almacenamiento de Fotos del Menú)

1. Función: Repositorio en la nube y Content Delivery Network (CDN) para optimizar y servir las fotos de los platos.
2. Variables requeridas: CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET.
3. Pasos del desarrollador: Registrarse en cloudinary.com, acceder al Dashboard y copiar las tres credenciales de API.
4. Comprobación: Ejecutar una petición POST multipart/form-data al endpoint /api/upload adjuntando un archivo de imagen. Debe retornar HTTP 200 con la propiedad imageUrl.

6.3. EmailJS (Servicio de Despacho de Recibos HTML)

1. Función: Envío de comprobantes de pago digitales formateados en HTML hacia las casillas de correo de los clientes.
2. Variables requeridas: EMAILJS_SERVICE_ID, EMAILJS_TEMPLATE_ID, EMAILJS_USER_ID, EMAILJS_ACCESS_TOKEN.
3. Pasos del desarrollador: En emailjs.com, vincular un Email Service (ej. Gmail o SMTP), crear un Template con variables dinámicas ({{cliente_nombre}}, {{tabla_items}}, {{total_pagado}}) y en Security habilitar "Allow EmailJS API requests" generando una clave privada (Access Token).
4. Comprobación: Enviar un recibo vía POST /api/pagos/:id/Enviar-recibo y constatar la recepción del mensaje.

6.4. QRServer API (Generación Pública de Códigos QR)

1. Función: Renderizado dinámico de imágenes QR estáticas que codifican el ID de la comanda y el monto a cobrar.
2. Variables requeridas: No requiere llaves privadas ni variables de entorno; es una invocación HTTPS pública.
3. Funcionamiento: En src/services/pago.service.ts el método generarPagoQR() compone la URL: `https://api.qrserver.com/v1/create-qr-code/?size=250x250&data=SABOR_GESTION_ID_<ID>_TOTAL_<TOTAL>`.
4. Comprobación: Invocar POST /api/pagos/generar-qr/:pedidoId; la respuesta retorna la propiedad qrUrl legible por cualquier visor de imágenes.

7. ESTRUCTURA DEL PROYECTO

El código fuente del backend se organiza bajo una estructura modular de cuatro capas concéntricas situada en el directorio `src/`, complementada por herramientas de mantenimiento en `scripts/` y documentación en `docs/`:

Árbol Físico de Directorios y Archivos del Repositorio

```
sabor-gestion-backend-personal/
% % % docs/                                # Contratos y documentación técnica
% % % % openapi.yaml                       # Especificación OpenAPI 3.0.3 (47 endpoints)
% % % % informe_funcional_backend.md       # Informe funcional del sistema
% % % % informe_incidentes_y_correcciones... # Bitácora de incidentes BUG-01 a BUG-05
% % % % Informe_Modulos_Backend_Sabor_Ges... # Documento académico formal
% % % scripts/                             # Scripts de mantenimiento y migración
% % % % fixUbicacionNames.ts               # Normalización de índices legacy de zonas
% % % % migrateUbicaciones.ts              # Sincronización de esquema de ubicaciones
% % % src/                                 # Código fuente del backend en TypeScript
% % % % configs/                           # Infraestructura y clientes externos
% % % % % cloudinary.ts                    # Instancia SDK Cloudinary y Multer memory storage
% % % % % db.ts                           # Conexión persistente Mongoose a MongoDB Atlas
% % % % controllers/                      # Capa de controladores REST (10 controladores)
% % % % middlewares/                      # Interceptores de seguridad y subida
% % % % % auth.middleware.ts               # Extracción y verificación de JWT Bearer (8h)
% % % % % rol.middleware.ts                # Autorización estricta por roles (RBAC)
% % % % % upload.middleware.ts             # Filtro de archivos Multer (5MB, imágenes)
% % % % models/                           # 11 Esquemas Mongoose de la base de datos
% % % % repositories/                     # Abstracción de acceso a datos (Query Builders)
% % % % routes/                           # Definición de rutas HTTP Express (47 rutas)
% % % % services/                          # Lógica de negocio y reglas transaccionales
% % % % socket/                           # Servidor Socket.IO, salas y eventos en vivo
% % % % types/                             # DTOs, interfaces de contratos y respuestas
% % % % utils/                             # Utilidades, semillas y normalización horaria
% % % % app.ts                             # Configuración de Express, CORS y middlewares
% % % % server.ts                          # Punto de entrada: Servidor HTTP + WebSockets + DB
% % % package.json                         # Metadatos, dependencias y scripts de ejecución
% % % pnpm-lock.yaml                       # Bloqueo de dependencias de pnpm
% % % tsconfig.json                       # Configuración del compilador TypeScript
% % % vercel.json                          # Configuración para despliegue Serverless en Vercel
```

7.1. Flujo de Control en la Arquitectura por Capas

Toda interacción entrante atraviesa una estricta cadena unidireccional: la petición HTTP arriba a Routes, pasa por Middlewares (verificación de token y roles), es recibida por Controllers (que validan parámetros), delega la regla a Services (donde residen las transiciones y cálculos matemáticos), interactúa con Repositories (que aplican `runValidators: true`), consulta los Models de Mongoose y finalmente persiste en MongoDB Atlas. En paralelo, Services emite señales de Socket.IO hacia las salas lógicas de salón y cocina.

8. IMPLEMENTACIÓN FUNCIONAL POR MÓDULOS

A continuación se desglosan los 10 módulos funcionales que conforman el prototipo. Cada módulo delimita su responsabilidad técnica, componentes de software involucrados e interacciones con el resto del sistema.

M01 — Autenticación y Control de Acceso

Propósito: Validación de identidad de los colaboradores y expedición de credenciales criptográficas stateless.

Funcionamiento: El controlador `usuario.controller` recibe `username` y `password` en `POST /api/usuarios/login`. El servicio `usuario.service` localiza al usuario en la colección `usuarios`, verifica que `estado` sea `true` y compara el hash mediante `bcryptjs.compare()`. Si es válido, firma un token JWT con expiración de 8 horas inyectando `id` y `rol`.

Componentes: `usuario.routes.ts`, `usuario.controller.ts`, `usuario.service.ts`, `Usuario.ts`, `auth.middleware.ts`.

Relación: Provee el token JWT indispensable para autenticar peticiones en el resto de los módulos y en el handshake de WebSockets.

M02 — Administración de Empleados

Propósito: Gobierno del ciclo de vida del personal del restaurante y arqueo de turnos de caja.

Funcionamiento: Expone operaciones CRUD restringidas al rol `Administrador`. Incluye el endpoint `PATCH /api/usuarios/:id/estado` que permite suspender o reactivar empleados. Si el empleado desactivado es un `Cajero` y adjunta un balance, se asienta de inmediato un documento en la colección `CierreCaja`.

Componentes: `usuario.controller.ts`, `usuario.service.ts`, `Usuario.ts`, `CierreCaja.ts`, `rol.middleware.ts`.

Relación: Integra una regla transaccional crítica que impide eliminar o suspender al último cajero activo del sistema gastronómico.

M03 — Gestión de Categorías y Menú

Propósito: Estructuración de la carta gastronómica, fijación de precios oficiales y fotografía de platos.

Funcionamiento: Administra familias de alimentos (`Entradas`, `Segundos`, `Bebidas`) y productos. La subida de imágenes en `POST /api/upload` almacena el archivo en memoria mediante `Multer` y lo transmite por streaming al SDK de `Cloudinary`. La disponibilidad del plato se maneja con el booleano `disponible` en `Plato.ts`.

Componentes: `categoria.routes.ts`, `plato.routes.ts`, `upload.routes.ts`, `Plato.ts`, `Categoria.ts`, `cloudinary.ts`.

Relación: Provee los precios oficiales que el módulo de pedidos congela al emitir comandas, impidiendo manipulaciones.

M04 — Disponibilidad Culinaria e Ingredientes

Propósito: Validación técnica preventiva de la preparación de platos en función del stock de insumos de cocina.

Funcionamiento: Administra ingredientes (con unidades de medida) y recetas (escandallos). Antes de registrar un pedido, el motor `validarDisponibilidadIngredientes()` cruza los platos con sus recetas. Si un ingrediente requerido tiene `disponible: false`, el pedido es rechazado (`HTTP 400`). Soporta exclusión semántica ante observaciones como "sin cebolla".

Componentes: `inventario.routes.ts`, `inventario.controller.ts`, `inventario.service.ts`, `Ingrediente.ts`, `Receta.ts`.

Relación: Actúa como escudo de calidad impidiendo que los meseros comprometan platos con insumos agotados.

M05 — Ubicaciones y Mesas

Propósito: Gestión del plano físico del establecimiento y estado de ocupación de mesas en salón.

Funcionamiento: Define sectores físicos (`Salón Principal`, `Terraza`, `VIP`) y mesas asociadas con numeración única. El endpoint `PATCH /api/mesas/:id/estado` permite transicionar entre `Libre`, `Ocupada` y `Cuenta Solicitada`. La actualización aplica `runValidators: true` para rechazar valores ajenos al enum y emite el evento `mesas:updated` vía WebSockets.

Componentes: `mesa.routes.ts`, `ubicacion.routes.ts`, `mesa.service.ts`, `Mesa.ts`, `Ubicacion.ts`, `socket.ts`.

Relación: Vincula los pedidos activos con el mobiliario y orquesta las alertas visuales de ocupación.

M06 — Gestión de Pedidos

Propósito: Apertura, fijación de precios, modificación y control de comandas en mesa.

Funcionamiento: Al ejecutar `POST /api/pedidos`, el servicio valida ingredientes, congela los precios de Plato en el detalle, genera un correlativo atómico `PED-XXXX` mediante `Contador.ts`, actualiza la mesa a `Ocupada` y emite `cocina:nuevo_pedido`. Permite añadir platos con `PUT /api/pedidos/:id` (reabriendo el pedido a `ABIERTO` si estaba en `ENTREGADO`) y solicitar cuenta con `PATCH /:id/solicitar-cuenta`.

Componentes: `pedido.routes.ts`, `pedido.controller.ts`, `pedido.service.ts`, `Pedido.ts`, `Contador.ts`, `Mesa.ts`.

Relación: Es el núcleo del flujo gastronómico; alimenta el tablero de cocina y transfiere la comanda a caja para cobro.

M07 — Preparación y Seguimiento de Pedidos

Propósito: Coordinación del trabajo en fogones y emisión de avisos de despacho hacia el salón.

Funcionamiento: Los cocineros consultan comandas activas y avanzan su estado mediante `PATCH /api/pedidos/:id/estado`. La máquina de estados admite: `ABIERTO` -> `EN_PREPARACION` -> `ENTREGADO`. Al llegar a `ENTREGADO`, se emite el evento `mesas:alerta_listo` a la sala `room:meseros` activando un timbre sonoro en la terminal del mesero. El estado `CERRADO` queda reservado a caja.

Componentes: `pedido.service.ts` (actualizarEstadoService), `socket.ts` (`room:meseros`).

Relación: Coordina los tiempos de despacho y minimiza demoras entre la cocina y la atención al cliente.

M08 — Gestión de Pagos y Cierre de Cuentas

Propósito: Liquidación financiera autoritativa, aplicación de descuentos, propinas y liberación de mesa.

Funcionamiento: El cajero ejecuta `POST /api/pagos/:pedidoId/procesar`. El servicio `pago.service` recalcula el subtotal oficial ignorando cualquier subtotalCierre enviado por el cliente. Aplica: $\text{total} = \text{subtotal} - \text{descuento} + \text{propina (sin IVA)}$. Ejecuta una sesión transaccional atómica en MongoDB: cambia el Pedido a `CERRADO`, crea el documento en Pago con estado `Pagado` y libera la Mesa a `Libre` emitiendo `mesas:updated`.

Componentes: `pago.routes.ts`, `pago.controller.ts`, `pago.service.ts`, `Pago.ts`, `Pedido.ts`, `Mesa.ts`.

Relación: Finaliza el ciclo transaccional de la comanda y reactiva la mesa para nuevos comensales.

M09 — Comprobantes y Pagos QR

Propósito: Generación de QR dinámico, simulación pedagógica de pagos móviles y despacho de recibos por email.

Funcionamiento: Permite generar la URL del QR con `QRServer API`. El webhook público `POST /api/pagos/notificar-qr/:pedidoId` actúa como simulación académica: emite `caja:solicitud_pago` a la sala `room:caja` para avisar al cajero sin liquidar dinero automáticamente. Una vez cobrado, `POST /:id/enviar-recibo` compila una plantilla HTML y la remite vía `EmailJS REST API`.

Componentes: `pago.service.ts`, `email.service.ts`, `facturaTemplate.ts`, `QRServer API`, `EmailJS`.

Relación: Conecta la experiencia de cobro digital con el soporte documental para el cliente.

M10 — Cierre de Caja y Dashboard

Propósito: Arqueo financiero consolidado por turnos y monitoreo gerencial en tiempo real.

Funcionamiento: `dashboard.service` ejecuta pipelines de agregación en MongoDB Atlas sobre pagos, pedidos, mesas y usuarios. Calcula 8 indicadores gerenciales clave: Ventas Totales, Pedidos Completados, Ticket Promedio, Desglose por Método de Pago, Platos Populares, Horas Pico, Ocupación de Mesas y Productividad por Mesero.

Componentes: `dashboard.routes.ts`, `dashboard.controller.ts`, `dashboard.service.ts`, `dashboard.repo.ts`.

Relación: Provee visibilidad administrativa para la toma de decisiones basada en datos operativos del restaurante.

9. EJECUCIÓN DEL BACKEND

El ciclo de vida del backend se administra mediante scripts estandarizados configurados en el archivo `package.json`. A continuación se detallan los modos de ejecución, compilación y carga de datos iniciales:

Comando pnpm	Script Interno	Propósito y Descripción Operativa
pnpm run dev	<code>nodemon --exec tsx src/server.ts</code>	Modo desarrollo con recarga automática en vivo ante cambios en TypeScript.
pnpm run build	<code>tsc</code>	Compila y transpila todo el código TypeScript generando los artefactos JS en <code>dist/</code> .
pnpm start	<code>node dist/server.js</code>	Ejecuta el servidor compilado en producción sobre el runtime nativo de Node.js.
pnpm run seed	<code>ts-node src/utils/seed.ts</code>	Puebla la base de datos con usuarios iniciales, categorías, platos y mesas base.
pnpm run seed:inventario	<code>tsx src/utils/seedInventario.ts</code>	Puebla ingredientes y recetas culinarias base (escandallos gastronómicos).
pnpm run lint	<code>eslint .</code>	Ejecuta el linter estático para validar reglas de sintaxis y buenas prácticas.
pnpm run format	<code>prettier --write .</code>	Aplica formateo automático consistente de indentación y estilo en todo el código.

9.1. Puesta en Marcha en Modo Desarrollo

Para iniciar el servidor en entorno local con recarga en caliente, ejecute:

Terminal — Inicialización del Servidor en Modo Desarrollo

```
$ pnpm run dev
> sabor-gestion-backend@1.0.0 dev
> nodemon --exec tsx src/server.ts

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] starting `tsx src/server.ts`
Base de Datos MongoDB Conectada: sabor_gestion_db
Servidor HTTP y WebSockets corriendo exitosamente en el puerto: 3000
Ambiente de ejecucion: development
CORS configurado para clientes: http://localhost:5173, https://quirquinita.onrender.com, https://tis-pied.vercel.app
```

9.2. Verificación de Disponibilidad (Health Check)

Una vez levantado el servidor, abra su navegador web o ejecute una llamada cURL hacia la URL base en el puerto 3000. El endpoint público `GET /api/health` debe retornar código HTTP 200 OK con el siguiente payload JSON:

Verificación del Endpoint de Salud (GET /api/health)

```
$ curl -i http://localhost:3000/api/health
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 95
Date: Sat, 03 Oct 2026 12:00:00 GMT
Connection: keep-alive

{
  "status": "success",
  "message": "API de Sabor & Gestión funcionando correctamente 0=pe"
}
```

10. PRUEBA INICIAL DEL SISTEMA (GUÍA DE VERIFICACIÓN)

Para asegurar que el backend se encuentra correctamente instalado, configurado y conectado con MongoDB Atlas sin necesidad de clientes frontend, se recomienda seguir el siguiente protocolo de prueba secuencial mediante cURL o Postman:

Paso 1: Inicio de Sesión de Empleado (Obtención del Token JWT)

Envíe una petición POST a `/api/usuarios/login` proporcionando credenciales válidas de un empleado existente (por ejemplo, cargado mediante el seed base):

Solicitud HTTP — Login de Personal

```
$ curl -X POST http://localhost:3000/api/usuarios/login \
-H "Content-Type: application/json" \
-d '{"username": "admin", "password": "password123"}'
```

Respuesta JSON — Token JWT Emitido

```
{
  "exito": true,
  "mensaje": "Inicio de sesión exitoso",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY3MDE5...\"",
  "usuario": {
    "id": "67019f2a89c1b4001a123456",
    "nombre": "Administrador",
    "apellido": "General",
    "rol": "Administrador",
    "email": "admin@sabor.com"
  }
}
```

Paso 2: Acceso a un Endpoint Protegido con Bearer Token

Utilice el token obtenido en el paso anterior dentro del encabezado `Authorization: Bearer <token>` para consultar la lista de mesas registradas (`GET /api/mesas`):

Solicitud HTTP — Consulta de Mesas Protegida

```
$ curl -X GET http://localhost:3000/api/mesas \
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
```

Respuesta JSON — Mesas Autenticadas

```
[
  {
    "_id": "6701a01b89c1b4001a654321",
    "numero": "Mesa 1",
    "capacidad": 4,
    "ubicacion": "Salón Principal",
    "estado": "Libre",
    "tipo": "normal"
  },
  {
    "_id": "6701a02289c1b4001a654322",
    "numero": "Mesa 2",
    "capacidad": 2,
    "ubicacion": "Terraza",
    "estado": "Ocupada",
    "tipo": "vip"
  }
]
```

Paso 3: Consulta del Catálogo Público de Platos

Compruebe que las rutas públicas operan sin requerir autenticación consultando el catálogo del menú gastronómico (`GET /api/platos`):

Solicitud HTTP — Catálogo de Platos Público

```
$ curl -X GET http://localhost:3000/api/platos
```

11. INTEGRACIÓN Y COMUNICACIÓN ENTRE COMPONENTES

El sistema opera bajo un paradigma reactivo híbrido: transacciones síncronas bajo el protocolo HTTP REST para operaciones deterministas de lectura/escritura, y eventos asíncronos bajo el protocolo WebSocket (Socket.IO) para la sincronización colaborativa inmediata entre las diferentes terminales del restaurante.

11.1. Arquitectura de Canales y Salas en Tiempo Real (Socket.IO)

Al iniciar la sesión, cada cliente establece una conexión WebSocket con el servidor mediante un handshake que valida el token JWT (src/socket/socket.ts). El servidor clasifica las conexiones en salas de difusión dirigidas (Rooms):

Topología de Salas Lógicas en Socket.IO	
Servidor Socket.IO (Handshake con verificación de JWT)	
%%% room:meseros	-> Alertas de platos despachados en cocina ('mesas:alerta_listo')
%%% room:caja	-> Alertas de cuentas solicitadas y pagos ('caja:solicitud_pago')
%%% user:{cajeroId}	-> Notificaciones exclusivas al cajero asignado a la comanda
%%% Difusión Global	-> Actualizaciones de mesas ('mesas:updated') y comandas a cocina

11.2. Catálogo de Eventos Reales de Socket.IO

Evento	Dirección	Origen en Backend	Sala Destino	Payload Transmitido
mesas:updated	Server -> Client	PATCH /mesas/:id/estado	Global	{ mesaId, estado, numero, capacidad }
cocina:nuevo_pedido	Server -> Client	POST /api/pedidos	Global (Cocina)	Comanda PED-XXXX y platos para preparación
cocina:actualizar_tablero	Server -> Client	PATCH /pedidos/:id/estado	Global (Cocina)	Comanda con estado actualizado
mesas:alerta_listo	Server -> Client	PATCH /pedidos/:id/estado	room:meseros	{ pedidoId, mesald, mesaNombre } (Timbre sonoro)
caja:solicitud_pago	Server -> Client	PATCH /pedidos/:id/solicitar	room:caja	Subtotal oficial, ítems y datos de comanda
mesas:pago_completado	Server -> Client	POST /pagos/:id/procesar	Global	{ mesald, mesaNombre, pedidoId, mensaje }
caja:pago_confirmado	Server -> Client	POST /pagos/notificar-qr/:id	room:caja	Aviso pedagógico de confirmación QR

11.3. Trazabilidad del Flujo Operativo Completo

La interacción integral del sistema gastronómico se desarrolla en 6 etapas rigurosamente orquestadas:

1. Apertura de Comanda: El Mesero ingresa la orden vía POST /api/pedidos. El backend valida insumos culinarios, congela precios, genera PED-XXXX, pasa la mesa a Ocupada y emite cocina:nuevo_pedido.
2. Preparación en Fogones: El Cocinero actualiza a EN_PREPARACION y luego a ENTREGADO vía PATCH /api/pedidos/:id/estado. El servidor emite mesas:alerta_listo a room:meseros activando una alerta sonora.
3. Solicitud de Cuenta: El Mesero ejecuta PATCH /api/pedidos/:id/solicitar-cuenta. La mesa pasa a Cuenta Solicitada y se transmite caja:solicitud_pago a los cajeros.
4. Liquidación y Cierre: El Cajero ejecuta POST /api/pagos/:id/procesar. El backend recalcula el subtotal en servidor, aplica descuentos y propinas (total = subtotal - descuento + propina), cambia el Pedido a CERRADO, genera el registro Pago en Pagado y libera la Mesa a Libre emitiendo mesas:updated.
5. Comprobante Digital: El Cajero remite el recibo HTML compilado al cliente mediante POST /:id/enviar-recibo vía EmailJS.
6. Arqueo y Dashboard: Al finalizar el turno, el Cajero cierra su balance registrando un documento en CierreCaja, visible en el Dashboard gerencial.

12. MEDIDAS DE SEGURIDAD Y REGLAS DE INTEGRIDAD

El backend incorpora múltiples barreras de defensa en profundidad auditadas y endurecidas durante las fases de control de calidad (Fases 16 a 17.6). A continuación se especifican las medidas técnicas implementadas:

12.1. Autenticación y Autorización Criptográfica

- **JSON Web Tokens Stateless:** Los tokens se firman mediante el algoritmo simétrico HMAC-SHA256 utilizando la clave secreta `JWT_SECRET`. Poseen una vigencia de 8 horas continuas (un turno gastronómico).
- **Hashing Seguro con bcryptjs:** Las contraseñas de los colaboradores se protegen con 10 rondas de salt antes de su persistencia; el sistema nunca almacena contraseñas en texto plano.
- **Control de Acceso RBAC Estricto:** El middleware `permitirRoles(...)` valida que el rol decodificado del token coincida con la lista permitida para cada ruta, respondiendo con HTTP 403 Forbidden antes de acceder al servicio.

12.2. Integridad Transaccional y Financiera en Backend

- **Autoridad Monetaria Centralizada (Solución BUG-03):** El subtotal y total de un cobro son recalculados exclusivamente en el servidor a partir de los ítems de la comanda o del `subtotalCierre` preexistente. Cualquier importe enviado en el cuerpo de la petición por el cliente es descartado.
- **Protección contra Mass Assignment (Solución BUG-02):** El endpoint de edición de pedidos (`PUT /api/pedidos/:id`) aplica una lista blanca estricta; ignora los campos financieros (`total`, `subtotal`, `subtotalCierre`) y prohíbe manipular directamente el campo `estado`.
- **Máquina de Estados Finita No Violable (Solución BUG-01):** La cocina sólo puede transicionar pedidos de ABIERTO a EN_PREPARACION y de allí a ENTREGADO. El estado CERRADO queda reservado a la caja tras el pago, y pedidos cancelados o cerrados son estrictamente inmutables.
- **Validación Estricta de Esquemas (Solución BUG-05):** Todas las actualizaciones en repositorios configuran `runValidators: true` en `findByIdAndUpdate`, impidiendo que se guarden valores ajenos al enum de Mesa (`Libre`, `Ocupada`, `Cuenta Solicitada`).
- **Protección del Último Cajero Activo:** El servicio de usuarios valida que siempre exista al menos un usuario activo con rol Cajero. Se impide eliminar o desactivar al único cajero disponible.
- **Aislamiento de la Simulación QR (Solución BUG-04):** El webhook `POST /api/pagos/notificar-qr/:pedidoId` es una notificación pública de simulación pedagógica; emite eventos WebSockets pero bajo ninguna circunstancia liquida fondos ni cierra el pedido.

12.3. Seguridad en Red y Archivos

- **Control de Orígenes Cruzados (CORS):** El servidor restringe el acceso a los orígenes autorizados: `localhost:5173`, `localhost:5174`, `https://quirquinita.onrender.com` y `https://tis-pied.vercel.app`.
- **Filtro de Archivos en Multer:** La subida de fotos de platos valida que el archivo pertenezca a `mimetypes` de imagen permitidos (`image/jpeg`, `image/png`, `image/webp`) y restringe el tamaño a un máximo de 5 MB.
- **Protección de Entorno en Seeds:** Los scripts `seed.ts` y `seedInventario.ts` verifican `process.env.NODE_ENV === "production"` y abortan la ejecución de inmediato si detectan un entorno productivo.

13. ESTRATEGIA Y CONFIGURACIÓN DE DESPLIEGUE

El repositorio incluye formalmente en su raíz el archivo `vercel.json`, configurado para empaquetar el servidor mediante el runtime `@vercel/node` y enrutar las peticiones como funciones Serverless:

`vercel.json` — Despliegue Nativo Serverless

```
{
  "version": 2,
  "builds": [
    {
      "src": "src/server.ts",
      "use": "@vercel/node"
    }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "dest": "src/server.ts"
    }
  ]
}
```

Consideración de Arquitectura de Despliegue

En entornos Serverless puros (como Vercel), las instancias son efímeras y no mantienen conexiones persistentes TCP, lo que limita el funcionamiento continuo del servidor Socket.IO. Para operaciones de producción completas que requieran WebSockets bidireccionales en vivo, se recomienda desplegar el proceso compilado de Node.js (`dist/server.js`) sobre plataformas con procesos persistentes en segundo plano (tales como Render, Railway o VPS Linux administrado con PM2 o systemd).

14. MANTENIMIENTO Y REPRODUCCIÓN DEL ENTORNO

14.1. Archivos Críticos que Deben Versionarse

- **package.json y pnpm-lock.yaml:** Definen el árbol estricto de dependencias y versiones de librerías utilizadas.
- **tsconfig.json:** Especifica las directivas del compilador TypeScript (target ECMAScript, module resolution y strictness).
- **docs/openapi.yaml:** Contrato estandarizado de la API RESTful (47 rutas HTTP tipadas) utilizable en Swagger UI o Postman.

14.2. Archivos Sensibles que Jamás Deben Compartirse

- **Archivo .env:** Contiene cadenas de conexión a base de datos y secretos privados de APIs.
- **Tokens de acceso y certificados:** Cualquier clave privada generada para proveedores de correo o nube.

14.3. Guía de Solución de Problemas Frecuentes (Troubleshooting)

- **Error ECONNREFUSED en MongoDB:** Causa: MONGO_URI inválida o la IP pública de la máquina no está en la lista blanca de Atlas. Solución: Ingresar a MongoDB Atlas -> Network Access y autorizar la IP actual.
- **Error 401 Unauthorized en Peticiones:** Causa: El token JWT ha expirado (más de 8 horas) o el formato del encabezado no incluye el prefijo Bearer. Solución: Reautenticarse en `/api/usuarios/login` e incluir `Authorization: Bearer <token>`.
- **Bloqueo de CORS en Navegador:** Causa: El cliente web intenta consumir la API desde un puerto no registrado. Solución: Agregar la URL del cliente en el arreglo origin de `src/app.ts`.
- **Fallo en Subida de Fotos Cloudinary:** Causa: Credenciales de Cloudinary incorrectas o archivo supera los 5 MB. Solución: Verificar `CLOUDINARY_*` en el archivo `.env`.

15. CONCLUSIONES

La implementación del prototipo backend para el sistema "Sabor & Gestión" representa la culminación exitosa de una arquitectura de software robusta, moderna y orientada al cumplimiento riguroso de reglas de negocio en el rubro gastronómico. A través de este manual y del trabajo realizado en el repositorio, se han consolidado los siguientes logros:

- **Entorno Determinista y Altamente Reproducible:** Gracias al uso riguroso de TypeScript 5.8.3, Node.js >=20.x y pnpm 12.4.2 con pnpm-lock.yaml, cualquier integrante del equipo o evaluador técnico puede reproducir el entorno operativo en cuestión de minutos sin divergencias de dependencias.
- **Arquitectura Desacoplada y Mantenible:** La división en cuatro capas (Routes, Controllers, Services, Repositories/Models) garantiza que las reglas de negocio permanezcan completamente aisladas de los protocolos de transporte y de los drivers de base de datos, facilitando futuras refactorizaciones y extensiones.
- **Integración Dual Síncrona y Asíncrona:** La combinación armónica de una API RESTful exhaustiva (47 endpoints) con un servidor de WebSockets estructurado en salas lógicas (Socket.IO) resuelve eficientemente la necesidad de sincronización en tiempo real de pedidos y mesas entre salón, cocina y caja.
- **Seguridad y Autoridad Financiera Blindadas:** El endurecimiento aplicado frente a vulnerabilidades críticas (BUG-01 a BUG-05) garantiza la autoridad matemática absoluta del servidor sobre los precios, el control estricto de acceso basado en roles (RBAC) y la integridad de las transacciones financieras multi-documento.
- **Madurez para Integración con Clientes Frontend:** El backend se entrega auditado, documentado bajo el estándar OpenAPI 3.0.3, con validación de tipos estricta y con todas sus rutas verificadas, constituyendo una plataforma sólida, escalable y lista para interoperar con interfaces de usuario web y terminales móviles.

Dictamen de Ingeniería y Entrega Formal

El prototipo backend documentado en este informe cumple a cabalidad con los requerimientos técnicos y funcionales de la sección "10. Implementación del Prototipo", habiendo superado las auditorías de consistencia, arquitectura, seguridad y reproducibilidad.